Part2 of Midterm Exam
EGRE246
Complex Numbers

1. Overview

Write the C ADT module Complex to implement complex numbers according to the specification given below.

File complex.h is defined as follows (you can download the file).

```
#ifndef COMPLEX_H
#define COMPLEX_H
#include <math.h>

typedef struct ComplexType *complex;

complex CreatComplex(double real, double imag);
void DestroyComplex(complex c);
double GetReal(const complex c);
double GetImaginary(const complex c);
double ComplexMagnitude(const complex c);
complex AddComplex(const complex c1, const complex c2);
complex SubtractComplex(const complex c1, const complex c2);
complex MultiplyComplex(const complex c1, const complex c2);
complex DivideComplex(const complex c1, const complex c2);
complex ConjugateComplex(const complex c);
complex SquareComplex(const complex c);
void DisplayComplex(const complex c1);
#endif
```

You may not modify file complex.h in any way. Note that you must define complexType in your C implementation file as a structure (complex.c). A complex number includes real and imaginary parts.

Notes on the program:

1. *CreatComplex* that takes two doubles and returns a pointer to a complex number type object allocated dynamically on the heap;
2. DestroyComplex deallocates the memory for a complex number;
3. ComplexMagnitude calculates the magnitude of a given complex number;
4. ConjugateComplex return a conjugate of a complex number by changing the sign of the imaginary part. You need to create a new complex number and return this complex number. For example, the conjugate of (3+2i) is (3-2i);
5. SquareComplex caculates the square of a complex number;
6. You only need to implement all these routines defined in the header file.

2. Suggested Program Development

Start by defining your data structure(s) and coding your implementation. Note that how you define your data structure(s) will strongly influence how you will need to write some of your functions. Test and implement any new function you write completely before moving on to additional routines. Always remember that any routine defined in your module can be used in any of the other routines.

3. Submission

You should only turn in your implementation file (complex.c)! I will test your code with my own driver test program(s). Note you need not turn in an executable file or your driver program!

Grading (4 points for each subroutine and 2 points for the data structures, 50 points in total). Please make sure that your code can pass the compilation check. Otherwise, the minimal penalty will be 20 points if your code cannot run with the test driver program.